Adopter Portal API Reference

This document defines the adopter-facing endpoints delivered in Sprint 3: profile management, government ID verification, and adoption application submission. It follows the same conventions established in 08-Public_API_Reference — see that document for base URL, standard response structure, and error code definitions, which are not repeated here.

1. Profile Endpoints

1.1 GET /api/v1/adopters/me

Returns the full profile of the currently logged-in adopter, including all schema fields from the ADOPTER table except those explicitly excluded for security.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "adopterID": 12,
    "adopterName": "...",
    "adopterEmail": "...",
    "housingType": "...",
    "ownsOrRents": "...",
    "householdSize": 3,
    "...": "...all other ADOPTER fields except excluded ones"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Adopter record does not exist	NOT_FOUND	404

> *Excluded fields: adopterPassword and stripeCustomerID are never included in this response, regardless of role — this mirrors the NF-10 data privacy requirement.*

1.2 PUT /api/v1/adopters/me

Allows the logged-in adopter to update their profile fields. Accepts a partial body — only fields present in the request are updated. Password changes are out of scope for this endpoint.

Request

Field	Value
Method	PUT
Auth required	Yes — Adopter only
Body fields	Any subset of updatable ADOPTER fields (see design note)

> *Enum fields are validated against their allowed values before the update is applied: housingType, ownsOrRents, employmentStatus, activityLevel, preferredSize, preferredAgeRange, adopterType. An invalid enum value returns VALIDATION_ERROR before any write occurs.*

Response

```
200 OK
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "...": "same shape as GET /adopters/me" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Invalid enum value in body	VALIDATION_ERROR	422
Attempt to update a disallowed field	VALIDATION_ERROR	422
Adopter record does not exist	NOT_FOUND	404

> *Disallowed fields: adopterEmail, adopterPassword, adopterRiskFlag, preQualifyFlag, and accountStatus cannot be updated through this endpoint — these are either admin-controlled, security-sensitive, or reserved for the AI matcher (Sprint 6). Any of these present in the request body are silently ignored, not written.*

2. Government ID Verification

These endpoints handle identity document upload and status retrieval for adoption-critical identity verification (Requirement A-01). Files are stored in Supabase Storage; only metadata is persisted in the GOVERNMENT_ID table.

2.1 POST /api/v1/adopters/me/government-id

Uploads a government ID document for identity verification. Stores file metadata in the GOVERNMENT_ID table; the file itself is uploaded to the government-ids/ bucket in Supabase Storage.

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Content-Type	multipart/form-data
Body fields	idType (string, required), idNumber (string, required, ≤45 chars), file (JPEG/PNG/WebP/HEIC/PDF, ≤5 MB, required)

Response

```
201 Created
{
  "success": true,
  "message": "Government ID submitted successfully",
  "data": {
    "governmentIDID": 7,
    "userID": 12,
    "userType": "Adopter",
    "idType": "Driver's License",
    "idNumber": "*****4567",
    "verificationStatus": "Pending"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing idType, idNumber, or file	VALIDATION_ERROR	422
File exceeds 5 MB or wrong type	BAD_REQUEST	400
Government ID already submitted for this adopter	CONFLICT	409

> *Response masking: idNumber is masked in the response (e.g. *****4567) — this is a deliberate current design choice, not a placeholder for a future fix. Revisit only if a product requirement emerges for the adopter to view their full submitted ID number.*

> *Uniqueness enforcement: duplicate submissions are prevented at the database level via a unique constraint — @@unique([userID, userType]) — on GOVERNMENT_ID, not solely by an application-level pre-check. A pre-check (findFirst) may still run first as an optimization to avoid an unnecessary file upload in the common case, but it is not the source of truth for the 409 — the Prisma unique-constraint error (P2002) on the insert is. This closes a race condition where two near-simultaneous requests could both pass a pre-check before either had written its row. On a P2002 error, the just-uploaded Storage file is deleted before the 409 response is returned, to avoid leaving an orphaned file.*

2.2 GET /api/v1/adopters/me/government-id

Returns the current adopter's government ID verification status.

Request

Field	Value

Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Government ID status retrieved successfully",
  "data": {
    "idType": "Driver's License",
    "verificationStatus": "Pending",
    "submittedAt": "2026-08-01T10:15:00Z"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
No government ID submitted yet	NOT_FOUND	404

> *Response scope: idNumber and the Storage file URL/path are never returned by this endpoint, regardless of role — only idType, verificationStatus, and submission date are exposed. This is stricter than the POST response, which masks but still includes idNumber.*

3. Adoption Applications

3.1 POST /api/v1/adoption-applications

Submits a new adoption application for a specific pet. Application status starts as Pending and is later updated by shelter staff (see PATCH /adoption-applications/:id/status, Sprint 4).

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Body fields	petID (int, required), shelterID (int, required)

> *shelterID in the request body is expected to match the pet's current shelterID — it identifies which branch the application is being processed at, consistent with the ADOPTION_APPLICATION schema.*

Response

```
201 Created
{
  "success": true,
  "message": "Application submitted successfully",
```

```
  "data": {
    "applicationID": 44,
    "petID": 9,
    "adopterID": 12,
    "shelterID": 3,
    "applicationStatus": "Pending",
    "createdAt": "2026-08-03T09:00:00Z"
  }
}
```


Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing petID or shelterID	VALIDATION_ERROR	422
Pet does not exist	NOT_FOUND	404
Pet adoptionStatus is not 'available'	CONFLICT	409
Adopter already has an active application for this pet	CONFLICT	409

> *Duplicate application check: this endpoint currently uses an application-level check (existing active application for the same adopter + pet) before inserting. Unlike the government-ID endpoint, there is no database-level uniqueness constraint backing this check yet — ADOPTION_APPLICATION has no @@unique on (adopterID, petID). This is a narrower race window than the government-ID case (a duplicate application is a data-quality issue, not an identity-integrity one), but worth the same fix if it becomes a priority: either a partial unique index scoped to Pending/Accepted statuses, or an equivalent application-level guard with idempotency handling.*